

Taller de Lenguajes de Programación

Sesión 02:

Introducción a Django



Introducción

Esta sesión cubre conceptos clave utilizados en Django, un framework web full-stack basado en Python. Si eres principiante en programación, puedes tener dificultades para entender algunos conceptos o términos. Sin embargo, es esencial que empieces a familiarizarte con esos nuevos conceptos o términos. Puedes hojear primero esta sección y volver a este capítulo más adelante, después de leer otros capítulos, que son más tangibles.

Objetivos de la sesión

- Comprender la naturaleza de Django y su filosofía fundamental de desarrollo.
- Crear y organizar la estructura de un proyecto Django desde sus cimientos.
- Entender el funcionamiento del patrón MVT y la arquitectura basada en aplicaciones (apps).
- Configurar adecuadamente las variables de entorno utilizando archivos `.env`.
- Contenedorizar la aplicación empleando Docker y establecer la conexión con PostgreSQL.
- Realizar la entrega del proyecto base del Sistema de Gestión de Encomiendas ejecutándose en Docker.

¿Qué es Django? Filosofía y ventajas

Django es un framework web de alto nivel escrito en Python que fomenta el desarrollo rápido y el diseño limpio y pragmático. Su lema es:

"The web framework for perfectionists with deadlines."

Filosofía DRY (Don't Repeat Yourself): Django promueve que cada pieza de conocimiento tenga una única representación en el sistema. Esto reduce la duplicación de código y facilita el mantenimiento.

Ventajas principales

- Batteries included: ORM, administrador automático, sistema de autenticación, formularios, migraciones — todo incluido sin instalar nada extra.
- Seguro por defecto: protección contra CSRF, XSS, SQL injection y clickjacking incluida de fábrica.
- Escalable: usado por Instagram, Disqus, Pinterest y Mozilla a escala masiva.

Gran comunidad: extensa documentación, paquetes de terceros (Django REST Framework, Celery, etc.).

Instalación y primer proyecto

Instalando Django

Ahora que hemos instalado y creado nuestro entorno virtual vamos a instalar Django como primer paso para empezar a trabajar con este framework.

1. Para instalarlo podemos ejecutar cualquiera de las siguientes instrucciones

None

```
pip install django
```

o

```
python -m pip install django
```

2. Se debe mostrar lo siguiente:

```
(env) D:\workspace\training\curso-python>pip install django
Collecting django
  Downloading Django-5.2-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref>=3.8.1 (from django)
  Using cached asgiref-3.8.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.3.1 (from django)
  Downloading sqlparse-0.5.3-py3-none-any.whl.metadata (3.9 kB)
Collecting tzdata (from django)
  Downloading tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
Downloading Django-5.2-py3-none-any.whl (8.3 MB)
----- 8.3/8.3 MB 24.4 MB/s eta 0:00:00
Using cached asgiref-3.8.1-py3-none-any.whl (23 kB)
Downloading sqlparse-0.5.3-py3-none-any.whl (44 kB)
Downloading tzdata-2025.2-py2.py3-none-any.whl (347 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.8.1 django-5.2 sqlparse-0.5.3 tzdata-2025.2

[notice] A new release of pip is available: 24.2 -> 25.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

3. Para validar la correcta instalación de Django, ejecutamos la siguiente instrucción en la línea de comandos

None

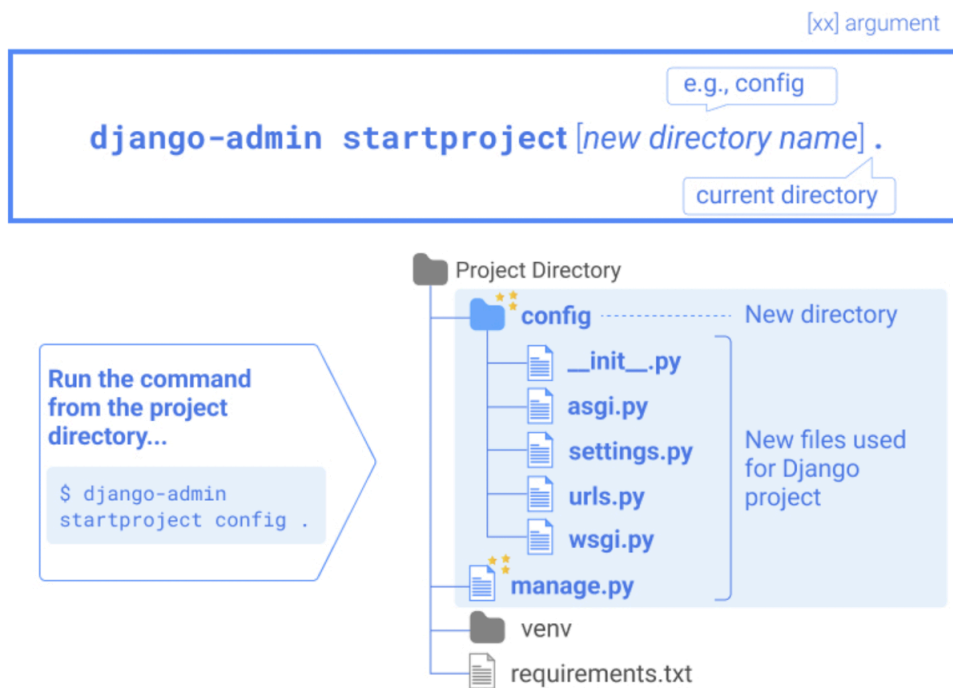
```
django-admin --version
```

Nos debe mostrar la versión instalada

```
(env) D:\workspace\training\curso-python\sesion02>django-admin --version
5.2
```

Creando nuestro primer proyecto con Django

A continuación, vamos crear nuestro primer proyecto con el cual vamos a empezar a dar nuestros primeros pasos con el framework

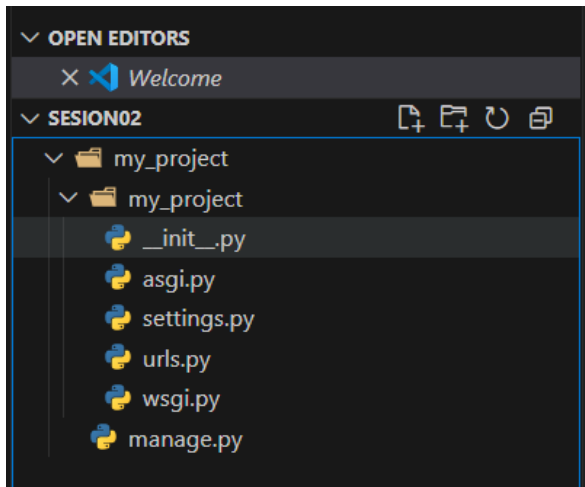


Crearemos nuestro proyecto llamado my_project:

Python

```
django-admin startproject my_project
```

Luego abrimos el proyecto en VS Code y observamos la siguiente estructura de carpetas:



Archivos de clave de Django creados con el comando `django-admin startproject`.

Al ejecutar el comando `django-admin startproject`, se crean varios archivos. A continuación, se ofrecen explicaciones sencillas de cada archivo.

`manage.py`

Este archivo se crea directamente en el directorio del proyecto, mientras que los demás archivos se crean en un nuevo directorio. El archivo `manage.py` se utiliza para ejecutar varios comandos de Django. Verá cómo usar este archivo en las siguientes secciones.

`__init__.py`

Este archivo específico para la programación en Python se utiliza para marcar un directorio como un paquete de Python. A menudo, no contiene contenido. En el uso habitual, no es necesario modificarlo.

`asgi.py`

Este módulo de Python define la aplicación ASGI (Interfaz de Puerta de Enlace de Servidor Asíncrona) para tu proyecto Django. Este archivo se utiliza al implementar una aplicación Django. Si eres principiante en Django, no necesitas preocuparte por este archivo.

`settings.py`



Al desarrollar una aplicación Django, este archivo se utiliza con frecuencia. Este archivo se utiliza para configurar la aplicación Django. Los detalles de settings.py se explicarán más adelante en este curso.

urls.py

Este archivo funciona como un administrador de URL de Django (un sistema de enrutamiento de URL). En él, se pueden definir patrones de URL que se conectarán con views.py; este último se utiliza para definir una respuesta cuando hay una solicitud HTTP con la URL especificada. Más adelante se explicará cómo editar urls.py.

wsgi.py

WSGI (Interfaz de Puerta de Enlace del Servidor Web) es una interfaz estándar que permite la comunicación entre aplicaciones web basadas en Python y servidores web como Apache y Nginx. wsgi.py es un módulo que implementa WSGI. Al igual que asgi.py, este archivo se utiliza al implementar una aplicación Django. *Si eres principiante en Django, no necesitas preocuparte por este archivo.*

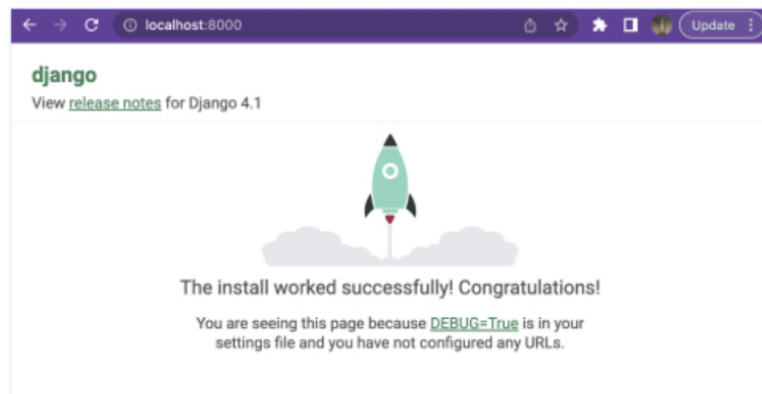
Ejecutando nuestro servidor web

```
python manage.py runserver
```

Run the command
from the project
directory...

```
$ python manage.py  
runserver
```

Type **localhost:8000** in a web browser



Django ofrece un servidor de desarrollo fácil de ejecutar para realizar pruebas durante el desarrollo de aplicaciones. Al ejecutar el comando runserver, la aplicación se implementa en el entorno local. El servidor ejecutado escucha en el puerto 8000. Puede acceder a la aplicación iniciada escribiendo localhost:8000 en su navegador web.

El comando runserver

Puedes ejecutar tu aplicación en tu entorno local rápidamente ejecutando el comando runserver. Aún no hemos desarrollado código, pero puedes comprobar su funcionamiento ejecutando el siguiente comando. El archivo manage.py se utiliza para llamar al comando runserver.

Python

```
python manage.py runserver
```

Cuándo ejecutemos el servidor se nos debe mostrar algo así

```
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin,
auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
April 13, 2025 - 00:41:27
Django version 5.2, using settings 'my_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
[13/Apr/2025 00:41:38] "GET / HTTP/1.1" 200 12068
Not Found: /favicon.ico
[13/Apr/2025 00:41:38] "GET /favicon.ico HTTP/1.1" 404 2212
```

Luego podemos acceder a sitio web copiando en nuestro navegador la dirección

<http://127.0.0.1:8000/>

← → ↻ 🔍 127.0.0.1:8000 Google Lens 📄 ☆



The install worked successfully! Congratulations!

View [release notes](#) for Django 5.2

You are seeing this page because `DEBUG=True` is in your settings file and you have not configured any URLs.

django

💡 **Django Documentation**
Topics, references, & how-to's

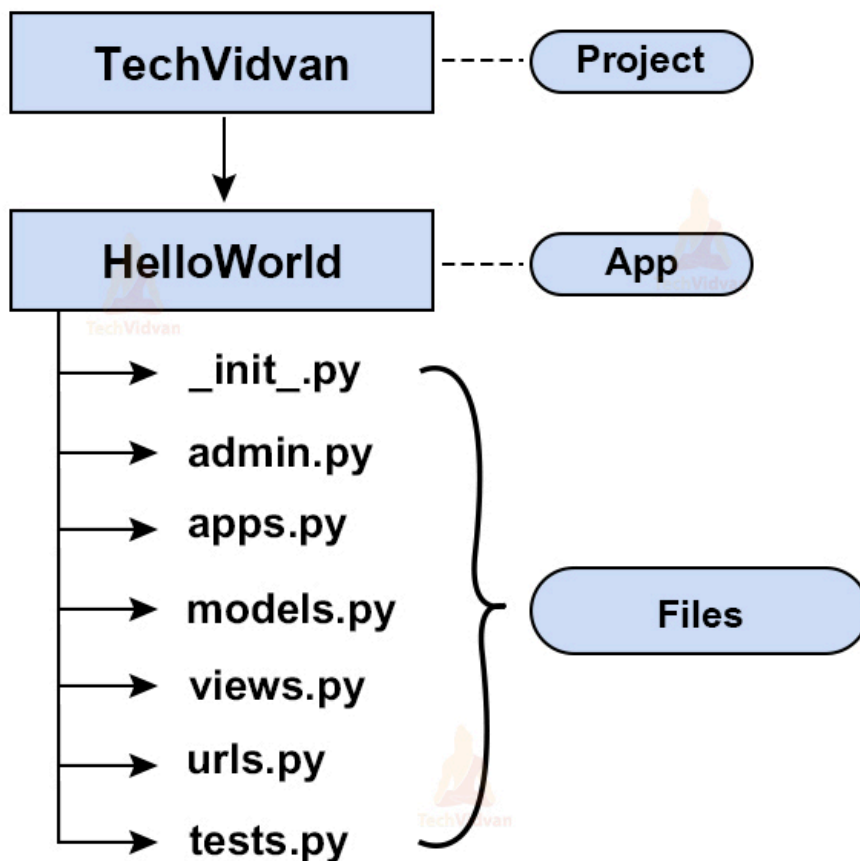
<> **Tutorial: A Polling App**
Get started with Django

👤 **Django Community**
Connect, get help, or contribute

Aplicaciones en Django

Django utiliza el concepto de **proyectos y aplicaciones** para gestionar el código y presentarlo en un formato legible. Un proyecto de Django contiene una o más aplicaciones que realizan el trabajo simultáneamente para garantizar un flujo fluido de la aplicación web.

Por ejemplo, un sitio web de comercio electrónico de Django real tendrá una aplicación para la autenticación de usuarios, otra para los pagos y una tercera para la información de los artículos; cada una se centrará en una sola funcionalidad.

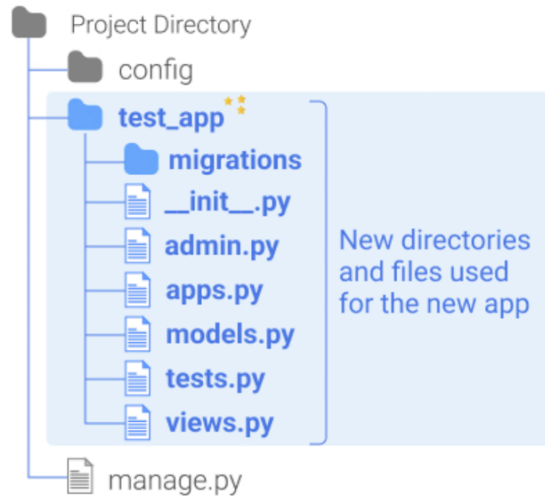


[xx] argument

```
python manage.py startapp [app name]
```

Run the command
from the project
directory...

```
$ python manage.py  
startapp test_app
```



Para crear o agregar aplicaciones dentro de nuestro proyecto django usa el siguiente comando

None

```
django-admin startapp <nombre_app>
```

o

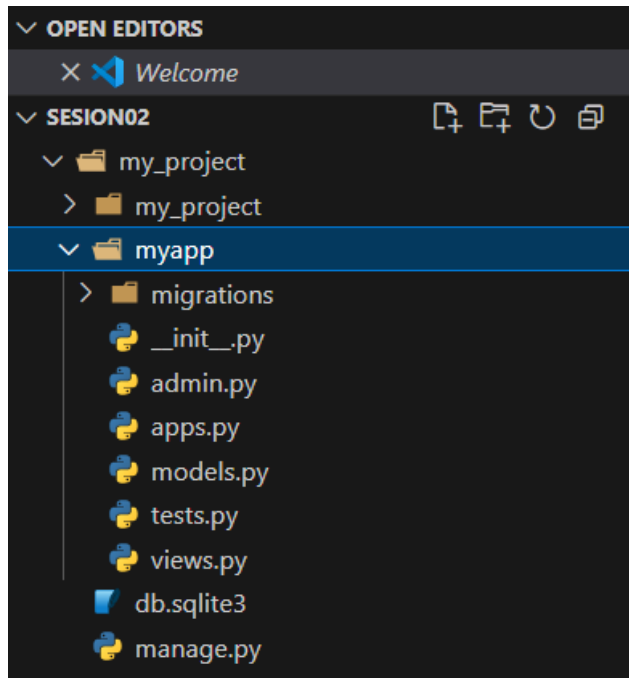
```
python manage.py startapp [nombre_app]
```

Para nuestro proyecto de ejemplo my_project, vamos a crear una aplicación llamada myapp, para ello ejecutamos la siguiente instrucción en nuestra línea de comandos

None

```
django-admin startapp myapp
```

Ahora vamos a VS Code y vemos que en nuestro proyecto se agregó una nueva carpeta llamada myapp



Directorio de migraciones

Al ejecutar el comando `startapp`, solo se crea el archivo `__init__.py` en este directorio. Este directorio se utiliza para almacenar los archivos de migración. Al ejecutar el comando `makemigration`, se crea un nuevo archivo de migración.

`__init__.py`

Como se explica en la sección "Iniciar proyecto Django", este es un archivo específico para la programación Python que se utiliza para marcar un directorio como un paquete Python. A menudo, este archivo no tiene contenido. En el uso habitual, no es necesario modificarlo.

`admin.py`



Este archivo se utiliza para personalizar el sitio de administración de Django con los datos creados por la aplicación.

apps.py

Este archivo se utiliza para personalizar la configuración de la aplicación. Si eres principiante en Django, no necesitas preocuparte por este archivo.

models.py (Very Important)

Este archivo se utiliza para diseñar la base de datos de la aplicación. En él, se pueden describir las estructuras de la base de datos, incluyendo la configuración de los campos y tipos de datos. Más adelante explicaremos cómo usar este archivo.

tests.py

Este archivo se utiliza para escribir y ejecutar pruebas para la aplicación Django.

views.py (Very Important)

Este archivo es uno de los más críticos en el desarrollo de aplicaciones Django. Se utilizará con frecuencia. Las funciones o clases para gestionar las solicitudes HTTP se diseñan a través de este archivo.

Registrando nuestra app

Al ejecutar únicamente el comando `startapp`, el proyecto Django no reconoce la nueva aplicación. Para registrar la aplicación recién creada, debe agregar su nombre en el archivo `settings.py`. En nuestro ejemplo, agregue `"test_app"` en la sección `INSTALLED_APPS` del archivo `settings.py` (que se muestra a continuación).

Abrimos el archivo `my_project/settings.py`

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'my_app'  
]
```

Creando el proyecto Django — Encomiendas

1. Crea una carpeta llamada **encomiendas**
2. Crearemos el proyecto base de nuestro **Sistema de Gestión de Encomiendas**:

None

```
# El punto (.) crea el proyecto en la carpeta actual
# Evita crear una carpeta adicional anidada
django-admin startproject config .
```

3. Vamos a verificar que el proyecto funcione, Levanta el servidor de desarrollo para comprobar que todo funciona:

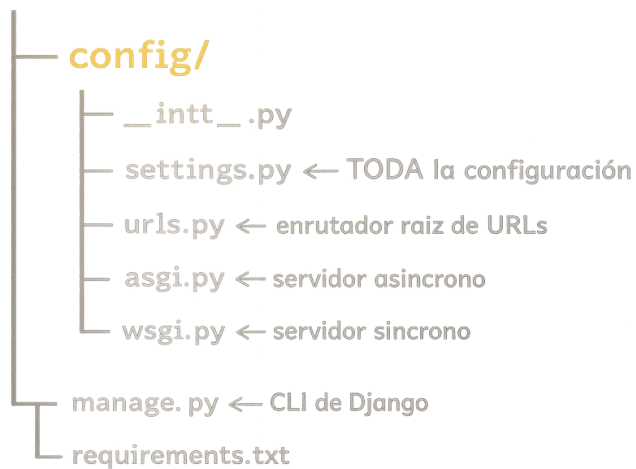
None

```
python manage.py runserver
# Abre en tu navegador: http://127.0.0.1:8000
# Deberías ver el cohete de Django.
```

Estructura del proyecto

Al ejecutar `django-admin startproject config` se genera la siguiente estructura de carpetas:

encomiendas/



Descripción de los archivos principales

manage.py — Tu panel de control. Desde aquí ejecutarás todos los comandos de Django:

None

```
python manage.py runserver          # inicia el servidor
python manage.py makemigrations     # crea archivos de migración
python manage.py migrate            # aplica migraciones a la BD
python manage.py createsuperuser    # crea administrador
python manage.py startapp nombre    # crea una nueva app
```


settings.py — Archivo de configuración global. Incluye: apps instaladas, configuración de base de datos, archivos estáticos, idioma, zona horaria, middlewares y mucho más.

urls.py — Mapa de URLs del proyecto. Conecta cada URL con su vista correspondiente.

wsgi.py / asgi.py — Puntos de entrada para el servidor web en producción. No se modifican durante el desarrollo.

__init__.py — Marca el directorio como paquete Python. No se modifica.

Patrón MVT (Model – View – Template)

Django sigue el patrón MVT, una variante del clásico MVC donde el framework actúa como el controlador:

Componente	Responsabilidad	En Encomiendas
Model	Estructura de datos y lógica de BD	Encomienda, Cliente, Ruta, Empleado
View	Lógica de negocio y procesamiento	Consultar estado, registrar envío
Template	Presentación HTML al usuario	Lista de encomiendas, detalle de envío

Flujo de una petición: Usuario → URL → View → consulta Model → renderiza Template → Respuesta HTTP

Apps en Django

Un proyecto Django se organiza en apps: módulos independientes y reutilizables. Cada funcionalidad principal del sistema vive en su propia app.

Regla práctica: una app debe hacer una cosa y hacerla bien. Por ejemplo: una app para envíos, otra para clientes, otra para reportes.

None

```
django-admin startapp envios
```

```
django-admin startapp clientes
```

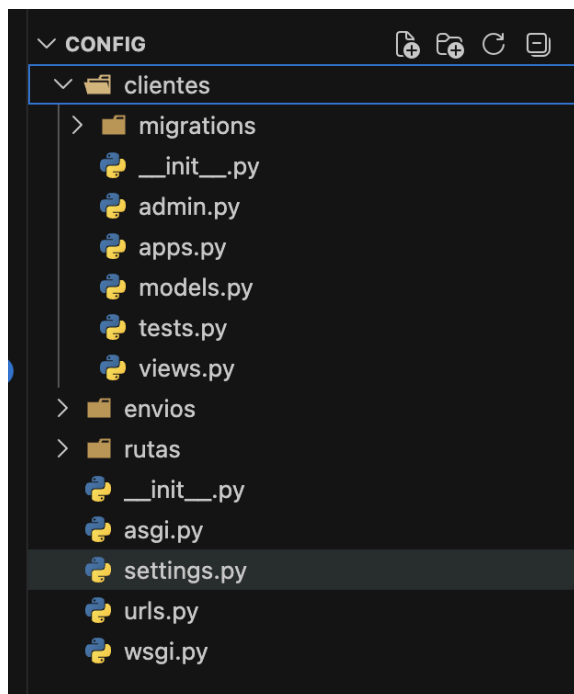
```
django-admin startapp rutas
```

```
python [python3] manage.py startapp envios
```

```
python [python3] manage.py startapp clientes
```

```
python [python3] manage.py startapp rutas
```

Estructura interna de cada app (ejemplo con la app clientes):



admin.py: Registra los modelos para que aparezcan en el panel de Django Admin.

models.py: Define la estructura de la base de datos. Uno de los archivos más importantes.

views.py: Define las funciones o clases que gestionan las peticiones HTTP.

migrations/: Carpeta automática con el historial de cambios en la base de datos.

Settings y Configuración

Registrando nuestras apps

Al crear las apps con `startapp`, Django no las reconoce automáticamente. Debemos registrarlas en `INSTALLED_APPS` dentro de `config/settings.py`:

Python

```
# config/settings.py
```

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    # Nuestras apps del proyecto encomiendas  
    'envios',  
    'clientes',  
    'rutas',  
]
```

Idioma y Zona Horaria

Python

```
# config/settings.py  
  
LANGUAGE_CODE = 'es-pe'  
  
TIME_ZONE = 'America/Lima'  
  
USE_I18N = True  
  
USE_TZ = True
```

Introducción a Docker

Docker es una plataforma que permite empaquetar una aplicación y todas sus dependencias en una unidad llamada contenedor, garantizando que funcione igual en cualquier máquina.

¿Por qué usar Docker en el proyecto?

El equipo desarrollará en Windows, Mac y Linux. Con Docker, todos corren exactamente el mismo ambiente: misma versión de Python, PostgreSQL y dependencias. "Funciona en mi máquina" deja de ser un problema.

Docker vs Máquina Virtual

Característica	Contenedor (Docker)	Máquina Virtual
Sistema Operativo	Comparte el kernel del host	SO completo virtualizado
Tiempo de arranque	Milisegundos	Minutos
Tamaño	Megabytes	Gigabytes
Rendimiento	Casi nativo	Menor rendimiento
Aislamiento	Proceso aislado	Completo

Imagen vs Contenedor

Imagen: plantilla de solo lectura definida en un Dockerfile. Es el 'molde' a partir del cual se crean contenedores.

Contenedor: instancia en ejecución de una imagen. Tiene su propio filesystem y ciclo de vida (crear, iniciar, detener, eliminar).

Docker en Proyectos Django

Laboratorio 2.3 — Containerizar el proyecto

Paso 1: Crear el Dockerfile

Crea un archivo llamado **Dockerfile** (sin extensión) en la raíz del proyecto:

```
None

# Dockerfile

# Imagen base oficial de Python
FROM python:3.11-slim

# Evitar archivos .pyc y buffering de stdout
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiar e instalar dependencias primero (aprovecha caché de capas)
COPY requirements.txt .
RUN pip install --upgrade pip && pip install -r requirements.txt

# Copiar el código fuente
COPY . .
```

```
# Puerto que expondrá el contenedor

EXPOSE 8000

# Comando por defecto al iniciar el contenedor

CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

Paso 2: Crear docker-compose.yml

Crea el archivo docker-compose.yml en la raíz del proyecto:

```
None

# docker-compose.yml

version: '3.9'

services:

  # Servicio de la app Django
  web:
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ../app          # monta el código en tiempo real
    ports:
```

```
- "8000:8000"

env_file:

- .env          # variables de entorno

depends_on:

- db

# Servicio de PostgreSQL

db:

  image: postgres:15-alpine

  volumes:

  - postgres_data:/var/lib/postgresql/data/

  environment:

  POSTGRES_DB: ${DB_NAME}

  POSTGRES_USER: ${DB_USER}

  POSTGRES_PASSWORD: ${DB_PASSWORD}

volumes:

  postgres_data:
```


Comandos esenciales de Docker Compose

None

Construir las imágenes

```
docker compose build
```

Levantar todos los servicios

```
docker compose up
```

Levantar en segundo plano (detached)

```
docker compose up -d
```

Ver logs del servicio web

```
docker compose logs -f web
```

Ejecutar un comando dentro del contenedor

```
docker compose exec web python manage.py migrate
```

Apagar todos los servicios

```
docker compose down
```

Variables de Entorno (.env)

Las variables de entorno mantienen datos sensibles (contraseñas, claves secretas) fuera del código fuente y del repositorio Git. Nunca deben subirse a GitHub.

Paso 1: Crear el archivo .env

Crea el archivo .env en la raíz del proyecto:

```
Python

# .env

# Django

SECRET_KEY=tu-clave-secreta-muy-larga-y-aleatoria

DEBUG=True

ALLOWED_HOSTS=localhost,127.0.0.1

# Base de datos PostgreSQL

DB_ENGINE=django.db.backends.postgresql

DB_NAME=encomiendas_db

DB_USER=encomiendas_user

DB_PASSWORD=encomiendas_pass_2026
```

Paso 2: Crear el .gitignore

Crea o edita .gitignore en la raíz para excluir archivos sensibles:

```
Python
```

```
# .gitignore
```

```
.env
```

```
venv/
```

```
env/
```

```
__pycache__/
```

```
*.pyc
```

```
db.sqlite3
```

```
.DS_Store
```

Buena práctica — .env.example

Crea un archivo `.env.example` con los nombres de las variables pero sin valores reales. Este archivo Sí se sube a Git para que otros desarrolladores sepan qué variables necesitan configurar.

```
SECRET_KEY=
```

```
DEBUG=True
```

```
DB_NAME=
```

```
DB_USER=
```

```
DB_PASSWORD=
```

Paso 3: Instalar python-decouple y leer el .env

```
None
```

```
pip install python-decouple
```

```
pip freeze > requirements.txt
```

Actualiza config/settings.py para leer las variables:

```
Python

# config/settings.py

from decouple import config

SECRET_KEY = config('SECRET_KEY')

DEBUG = config('DEBUG', cast=bool, default=False)

ALLOWED_HOSTS = config('ALLOWED_HOSTS', default='').split(',')
```

Django + PostgreSQL con Docker

Laboratorio 2.4 — Sistema de Encomiendas corriendo en Docker

Paso 1: Configurar la base de datos en settings.py

Reemplaza la configuración de SQLite por PostgreSQL leyendo los valores del .env:

```
Python

# config/settings.py

from decouple import config

DATABASES = {

    'default': {

        'ENGINE': config('DB_ENGINE'),

        'NAME': config('DB_NAME'),

        'USER': config('DB_USER'),

        'PASSWORD': config('DB_PASSWORD'),
```

```
'HOST': config('DB_HOST', default='localhost'),  
'PORT': config('DB_PORT', default='5432'),  
}  
}
```

Paso 2: Instalar el adaptador de PostgreSQL

```
Python  
  
pip install psycopg2-binary  
pip freeze > requirements.txt
```

Paso 3: Crear el primer modelo — Encomienda

Abre **envios/models.py** y crea el modelo:

```
Python  
  
# envios/models.py  
  
from django.db import models  
  
class Encomienda(models.Model):  
    class EstadoChoices(models.TextChoices):  
        PENDIENTE = 'PE', 'Pendiente'  
        EN_TRANSITO = 'TR', 'En tránsito'  
        ENTREGADO = 'EN', 'Entregado'
```

```

        DEVUELTO      = 'DE', 'Devuelto'

    codigo      = models.CharField(max_length=20, unique=True)
    descripcion  = models.TextField()
    peso_kg      = models.DecimalField(max_digits=8, decimal_places=2)
    estado       = models.CharField(
        max_length=2, choices=EstadoChoices.choices,
        default=EstadoChoices.PENDIENTE)
    fecha_envio   = models.DateTimeField(auto_now_add=True)
    fecha_entrega = models.DateTimeField(null=True, blank=True)

    def __str__(self):
        return f"{self.codigo} - {self.get_estado_display()}"

    class Meta:
        verbose_name      = 'Encomienda'
        verbose_name_plural = 'Encomiendas'
        ordering           = ['-fecha_envio']

```

Paso 4: Registrar en el Admin

Python

```

# envios/admin.py

from django.contrib import admin

```

```
from .models import Encomienda

@admin.register(Encomienda)
class EncomiendaAdmin(admin.ModelAdmin):

    list_display = ('codigo', 'descripcion', 'peso_kg', 'estado',
                    'fecha_envio')

    list_filter = ('estado',)

    search_fields = ('codigo', 'descripcion')
```

Paso 5: Levantar todo y aplicar migraciones

None

1. Construir y levantar los contenedores

```
docker compose up --build -d
```

2. Crear migraciones del modelo Encomienda

```
docker compose exec web python manage.py makemigrations
```

3. Aplicar migraciones a PostgreSQL

```
docker compose exec web python manage.py migrate
```

4. Crear superusuario para el admin

```
docker compose exec web python manage.py createsuperuser
```

5. Verificar en el navegador

App: <http://localhost:8000>

```
# Admin: http://localhost:8000/admin
```

Estructura final del proyecto

```
encomiendas/
├── config/
│   ├── settings.py <- BD con variables .env
│   └── urls.py
├── envios/
│   ├── models.py <- Modelo Encomienda
│   └── admin.py <- Registro en Admin
├── clientes/
├── rutas/
├── Dockerfile
├── docker-compose.yml
├── .env <- NO subir a Git
├── .env.example <- SÍ subir a Git
├── .gitignore
└── requirements.txt
```

Entregable de la Sesión 2

Al finalizar la sesión debes poder demostrar:

1. Proyecto Django con las 3 apps creadas (envios, clientes, rutas).
2. Aplicación corriendo en Docker con PostgreSQL (docker compose up).
3. Modelo Encomienda creado y migrado correctamente.
4. Acceso al panel de administración en <http://localhost:8000/admin>.
5. Archivo .env configurado y en .gitignore.

Repositorio publicado en GitHub con .env.example incluido.



Prox ima sesión: Modelos y ORM de Django — relaciones entre Encomienda, Cliente y Ruta.
Migraciones avanzadas y Django Shell.